

nerdtime.dev Specification

FULL-STACK RUST DEVELOPMENT & SYNC ENGINE

System Architecture: Low-friction, local-first workflow tracker. Built with a lightning-fast Rust CLI writing locally to SQLite, paired with a multi-tenant Loco web framework (Axum / SeaORM) running inside Docker behind Traefik on a VPS.

1. The Rust CLI Client (`nerd`)

The CLI is designed to run instantaneously in user terminal loops. It acts as an offline-first storage engine using SQLite to maintain focus session data before publishing up to the cloud node.

A. Local Storage Schema (SQLite in `~/config/nerdtime/data.db`)

```
CREATE TABLE IF NOT EXISTS sessions (  
  id TEXT PRIMARY KEY NOT NULL,  
  project_name TEXT NOT NULL,  
  branch_name TEXT,  
  commit_hash TEXT,  
  description TEXT,  
  started_at TEXT NOT NULL, -- ISO 8601 String  
  ended_at TEXT,           -- Null implies active session  
  is_synced INTEGER DEFAULT 0  
);
```

B. CLI Execution Model (using `clap` & `rusqlite`)

```
// src/main.rs (CLI structure example)
use clap::{Parser, Subcommand};
use rusqlite::{Connection, Result};
use chrono::Utc;
use uuid::Uuid;

#[derive(Parser)]
#[command(name = "nerd", version = "0.1.0", about = "Flow-first tracking")]
struct Cli {
    #[command(subcommand)]
    command: Commands,
}

#[derive(Subcommand)]
enum Commands {
    Start { project: String, #[arg(short, long)] desc: Option<String> },
    Stop,
    Status,
    Sync,
}

fn main() -> Result<()> {
    let conn = Connection::open("~/config/nerdtime/data.db")?;
    let cli = Cli::parse();

    match &cli.command {
        Commands::Start { project, desc } => {
            let id = Uuid::new_v4().to_string();
            let now = Utc::now().to_rfc3339();
            conn.execute(
                "INSERT INTO sessions (id, project_name, description, started_at) VALUES (?1, ?2, ?3, ?4)",
                (&id, project, desc, &now),
            )?;
            println!("Tracking started for project: {}", project);
        }
        Commands::Stop => {
            let now = Utc::now().to_rfc3339();
            conn.execute(
                "UPDATE sessions SET ended_at = ?1 WHERE ended_at IS NULL",
                [&now],
            )?;
            println!("Tracking stopped.");
        }
        _ => println!("Other commands executed."),
    }
    Ok(())
}
```

2. The Loco Backend App (`loco.rs`)

The cloud sync backend uses the **Loco** framework, mounting SeaORM and Axum. It handles secure token authentication, stores aggregate developer statistics inside PostgreSQL, and serves the central web metrics dashboard.

A. Database Model Migration (SeaORM in Loco)

```
// migration/src/m20260101_000001_sessions.rs
use loco_rs::schema::table_auto_tz;
use sea_orm_migration::prelude::*;

#[derive(DeriveMigrationName)]
pub struct Migration;

#[async_trait::async_trait]
impl MigrationTrait for Migration {
    async fn up(&self, manager: &SchemaManager) -> Result<(), DbErr> {
        manager
            .create_table(
                table_auto_tz(Sessions::Table)
                    .col(ColumnDef::new(Sessions::Id).uuid().not_null().primary_key())
                    .col(ColumnDef::new(Sessions::UserId).uuid().not_null())
                    .col(ColumnDef::new(Sessions::ProjectName).string().not_null())
                    .col(ColumnDef::new(Sessions::BranchName).string())
                    .col(ColumnDef::new(Sessions::CommitHash).string())
                    .col(ColumnDef::new(Sessions::Description).text())
                    .col(ColumnDef::new(Sessions::StartedAt).timestamp_with_time_zone().not_null())
                    .col(ColumnDef::new(Sessions::EndedAt).timestamp_with_time_zone())
                    .to_owned(),
            )
            .await
    }

    async fn down(&self, manager: &SchemaManager) -> Result<(), DbErr> {
        manager
            .drop_table(Table::drop().table(Sessions::Table).to_owned())
            .await
    }
}

#[derive(DeriveIden)]
enum Sessions {
    Table,
    Id,
    UserId,
    ProjectName,
    BranchName,
    CommitHash,
    Description,
    StartedAt,
    EndedAt,
}
```

B. Loco Sync Endpoint Controller

Handles incoming JSON batches pushed from the CLI client, performing atomic upsert loops to sync offline edits.

```

// src/controllers/sync.rs
use loco_rs::prelude::*;
use axum::Json;
use serde::{Deserialize, Serialize};

#[derive(Serialize, Deserialize)]
pub struct SyncPayload {
    pub id: String,
    pub project_name: String,
    pub started_at: String,
    pub ended_at: Option<String>,
}

pub async fn sync_sessions(
    State(_ctx): State<AppContext>,
    Json(payload): Json<Vec<SyncPayload>>,
) -> Result<Response> {
    // Process sync batch and save to Cloud PostgreSQL using SeaORM
    for session in payload {
        // Upsert operations logic goes here...
    }
    format::json("Sync successful")
}

```